

Unit-6: Concurrency Control (Threads & Synchronization)

Unit-6: Topics and Subtopics Table

Main Topic	Subtopics	What You Learn
1. Concurrency Fundamentals (OS Concepts)	Process	Program in execution
	Thread	Lightweight process
	Difference: Process vs Thread	Memory, speed, resource usage
	Subprocess concept	Child execution units
	Multitasking	Multiple processes
	Multithreading	Multiple threads in one process
	Concurrency vs Parallelism	Logical vs physical execution
2. Thread Basics in Java	Thread concept	Independent execution path
	Advantages of multithreading	Performance, responsiveness
	Single-thread vs Multi-thread	Execution comparison
3. Thread Life Cycle	New state	Thread created
	Runnable state	Ready to run
	Running state	Executing
	Waiting/Blocked state	Waiting for resource
	Terminated state	Execution finished
4. Creating Threads in Java	Using Thread class	Extending Thread class
	Using Runnable interface	Implementing Runnable
	Thread vs Runnable	Design comparison
5. Thread Methods	start()	Start thread execution
	run()	Thread logic
	sleep()	Pause execution
	join()	Wait for thread
	yield()	Give chance to other threads
	getName(), setName()	Thread identification
	isAlive()	Check thread status
6. Thread Synchronization	Need for synchronization	Avoid data inconsistency
	synchronized keyword	Control shared resources
	Synchronized method	Lock entire method

	Synchronized block	Lock specific part
7. Problems in Multithreading	Race condition	Multiple threads conflict
	Data inconsistency	Wrong results
	Deadlock (basic idea)	Threads waiting forever
8. Desynchronization Concept	Without synchronization	Unsafe execution
	Example of issue	Data corruption
9. Real-World Applications	Banking system	Multiple transactions
	Web servers	Multiple users
	Operating system	Process scheduling

Unit-6: Concurrency Fundamentals

This section builds the **base understanding of how programs run inside a computer system**. Without this, threads and synchronization will not be clear.

1. Process (Program in Execution)

Definition: A process is a program that is currently being executed by the CPU.

Deep Understanding

When you write code, it is just a **file (program)**.

When you run it, the operating system:

- loads it into memory
- assigns CPU time
- manages resources

🔗 At this moment, it becomes a **process**

Real Example You open Chrome → Chrome.exe starts → becomes a process

Internal Structure of Process

Process Memory Layout:

Code Section

Data Section

Heap (dynamic memory)

Stack (function calls)

Each process has:

- its own memory
- its own resources
- its own execution

Key Characteristics

Feature	Explanation
Independent	Runs separately
Heavyweight	Uses more memory
Own resources	CPU, memory, files
Isolation	Cannot directly access other processes

2. Thread (Lightweight Process)

Definition: A thread is a smaller unit of execution inside a process.

☞ It is called a **lightweight process**

Deep Understanding

A process can contain **multiple threads**, and all threads:

- share the same memory
- share the same resources
- run independently

Real Example

Chrome Process

- └ Tab 1 → Thread
- └ Tab 2 → Thread
- └ Download → Thread

Key Characteristics

Feature	Explanation
Lightweight	Less memory usage
Fast execution	Less overhead
Shared memory	Common resources
Dependent	Exists inside process

3. Difference: Process vs Thread

Detailed Comparison

Feature	Process	Thread
Definition	Program in execution	Part of process
Memory	Separate memory	Shared memory
Speed	Slower	Faster
Resource usage	High	Low
Communication	Difficult	Easy
Isolation	Strong	Weak
Creation cost	High	Low

Core Understanding

Process → Independent program

Thread → Part of program

4. Subprocess Concept (Child Execution Units)

Definition: A subprocess (child process) is a process created by another process (parent process).

Deep Understanding

When a process creates another process:

Parent Process → Child Process

Example: Command Prompt → runs Java program → creates new process

Key Idea

Term	Meaning
Parent process	Original process
Child process	Created process

Important Points

- Child process has its own memory
- Runs independently
- Can communicate with parent

5. Multitasking (Multiple Processes)

Definition: Multitasking is the ability of an operating system to run multiple processes at the same time.

Deep Understanding

Your system runs:

- Browser
- Music player
- IDE
- OS background tasks

All simultaneously.

Types of Multitasking

1. Process-based Multitasking

Multiple processes running

Example: Chrome + VS Code + Spotify

2. Time Sharing

CPU switches rapidly between processes:

Process A → Process B → Process C → repeat

Key Concept

Looks parallel → actually switching fast

6. Multithreading (Multiple Threads in One Process)

Definition: Multithreading is the ability of a program to run multiple threads within a single process.

Deep Understanding

Instead of creating multiple processes (heavy), we use threads (lightweight).

Example

Music App:

- └ Play music → Thread
- └ Download song → Thread
- └ UI update → Thread

Advantages

Advantage	Explanation
Faster execution	Less overhead
Efficient resource use	Shared memory
Better responsiveness	UI doesn't freeze

Parallel tasks	Multiple work at once
----------------	-----------------------

Important Insight

Multitasking → multiple processes

Multithreading → multiple threads in one process

7. Concurrency vs Parallelism

This is one of the **most important conceptual differences**.

Concurrency

Definition: Concurrency means multiple tasks are in progress at the same time, but not necessarily executing at the exact same moment.

Deep Understanding

CPU switches between tasks very fast:

Task A → Task B → Task C → repeat

☞ Gives illusion of simultaneous execution

Example

Single CPU system running multiple apps

Key Idea

Concurrency = Managing multiple tasks

Parallelism

Definition: Parallelism means multiple tasks are executed at the exact same time using multiple CPUs or cores.

Deep Understanding

CPU Core 1 → Task A

CPU Core 2 → Task B

CPU Core 3 → Task C

☞ True simultaneous execution

Example: Multi-core processor executing multiple threads

Key Idea: Parallelism = Executing multiple tasks simultaneously

Difference: Concurrency vs Parallelism

Feature	Concurrency	Parallelism
Execution	Not simultaneous	Simultaneous
CPU	Single or multiple	Multiple cores required
Goal	Manage tasks	Speed up execution
Example	OS task switching	Multi-core processing

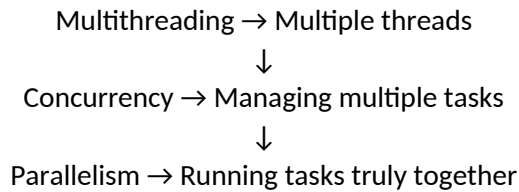
Final Mental Model (Very Important)

Process → Running program



Thread → Smaller execution unit





Real-World Combined Example

Online Video Streaming App

Process: Netflix App

- └ Thread 1 → Video streaming
- └ Thread 2 → Buffering
- └ Thread 3 → UI updates
- Multiple threads → multithreading
- CPU switching → concurrency
- Multi-core CPU → parallelism

Final Understanding

Process → Heavy, independent

Thread → Lightweight, shared

Multitasking → Multiple processes

Multithreading → Multiple threads

Concurrency → Manage many tasks

Parallelism → Execute many tasks

Unit-6: Thread Basics in Java

This section explains **what threads are, why they are needed, and how they improve program execution**. It builds directly on OS concepts like **process and concurrency** and is part of your concurrency control unit.

1. Thread Concept

Definition: A thread is the smallest unit of execution within a program (process).

☞ It represents a **single flow of control** inside a program.

Deep Understanding

When a Java program runs:

Program → Process → Threads

- The program becomes a **process**
- Inside that process, execution is divided into **threads**

Example

Simple Java program:

```
class Test {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
    }  
}
```

☞ This runs as **single thread** (main thread)

Multi-threaded Example Concept

Video Player App:

- └ Play video → Thread
- └ Load subtitles → Thread
- └ Handle UI → Thread

👉 Multiple tasks run independently

Internal Working

Each thread has:

Component	Explanation
Program counter	Current instruction
Stack	Method calls
Registers	CPU working data

👉 Threads **share memory (heap)** but have **separate stacks**

Key Characteristics

Feature	Explanation
Lightweight	Uses less memory
Fast creation	Compared to process
Shared memory	Common resources
Independent execution	Runs separately

Important Insight

Thread = Path of execution inside program

2. Advantages of Multithreading

Multithreading means **running multiple threads within a single program**.

Why Multithreading is Needed

Without multithreading:

Task 1 → Task 2 → Task 3 (sequential, slow)

With multithreading:

Task 1 + Task 2 + Task 3 (simultaneous/concurrent)

Detailed Advantages

1. Improved Performance

Multiple threads can run:

- at the same time (parallelism)
- or quickly switch (concurrency)

👉 Reduces total execution time

Example

Download + Play video at same time

2. Better CPU Utilization

CPU does not remain idle.

One thread waiting → another executes

3. Faster Execution

Instead of sequential execution:

$A \rightarrow B \rightarrow C$

We get:

$A + B + C$

4. Improved Responsiveness

User interface remains active.

Example

App is downloading file but still responding to clicks

5. Efficient Resource Sharing

Threads share:

- memory
- files
- data

🔗 No need to duplicate resources

6. Real-Time Applications

Used in:

Application	Usage
Games	Graphics + physics
Banking	Multiple transactions
Web servers	Handle multiple users
OS	Background tasks

7. Scalability

Programs can easily handle **multiple tasks/users**

Summary of Advantages

- ✓ Faster execution
- ✓ Better CPU usage
- ✓ Responsive UI
- ✓ Efficient resource usage
- ✓ Supports concurrent tasks

3. Single-Thread vs Multi-Thread

This is one of the **most important conceptual comparisons**.

Single-Threaded Program

Concept: Only **one thread executes at a time**

Flow

Task 1 → Task 2 → Task 3

Example

```
class Test {  
    public static void main(String[] args) {
```



```

        System.out.println("Task 1");
        System.out.println("Task 2");
        System.out.println("Task 3");
    }
}

```

Characteristics

Feature	Explanation
Sequential execution	One by one
Simple	Easy to understand
Slow	No parallel work
Blocking	One task blocks others

Multi-Threaded Program

Concept: Multiple threads run **simultaneously or concurrently**

Flow: Task 1 + Task 2 + Task 3

Example Concept

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

class Test {
    public static void main(String[] args) {
        MyThread t1=new MyThread();
        MyThread t2=new MyThread();
        t1.start();
        t2.start();
    }
}

```

Characteristics

Feature	Explanation
Concurrent execution	Multiple tasks
Faster	Reduced time
Complex	Requires synchronization
Non-blocking	Tasks run independently

Comparison Table

Feature	Single Thread	Multi Thread
Execution	Sequential	Concurrent
Speed	Slow	Fast
Complexity	Simple	Complex
Resource use	Less efficient	Efficient
Responsiveness	Poor	High
Failure impact	Stops program	Other threads continue

Deep Understanding

Single Thread: One worker doing all tasks

Multi Thread: Multiple workers doing tasks together

Real-World Example

Food Delivery App

Single-thread

Take order → Cook → Deliver → Repeat

Slow ✗

Multi-thread

Thread 1 → Take orders

Thread 2 → Cook food

Thread 3 → Deliver food

Fast ✓

Important Concept Link

Process → contains threads

Thread → executes tasks

Multithreading → multiple threads

Final Understanding

Thread → smallest execution unit

Multithreading → multiple threads in one program

Single-thread → one task at a time

Multi-thread → multiple tasks at same time

Goal:

- ✓ Improve performance
- ✓ Efficient CPU usage
- ✓ Better responsiveness

Unit-6: Thread Life Cycle

A thread does not start and end directly. It goes through **different stages (states)** during its execution. Understanding this lifecycle is essential to control thread behavior correctly.

1. What is Thread Life Cycle?

Definition: The thread life cycle is the sequence of states a thread passes through from creation to termination.

Overall Flow

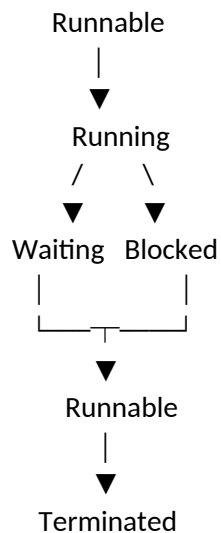
New → Runnable → Running → Waiting/Blocked → Runnable → Terminated

🔄 A thread can move **forward and sometimes back** (e.g., waiting → runnable)

Complete Diagram

New





2. New State

Definition: A thread is in **New state** when it is created but not yet started.

How It Happens

Thread t=new Thread();

☞ Thread object is created

☞ But it has not started execution

Key Points

Feature	Explanation
Thread created	Yes
Execution started	No
Memory allocated	Yes

Important Note

start() method not called yet

3. Runnable State

Definition: A thread enters **Runnable state** when **start()** is called, and it is ready to run.

How It Happens: t.start();

Deep Understanding

Runnable does NOT mean running immediately.

Runnable = Ready to run (waiting for CPU)

Key Points

Feature	Explanation
Thread ready	Yes
CPU assigned	Not yet
Waiting in queue	Yes

Important Concept

Multiple threads compete for CPU

4. Running State

Definition: A thread is in Running state when it is actively executing on CPU.

How It Happens

Scheduler selects thread from Runnable → moves to Running

Key Points

Feature	Explanation
CPU assigned	Yes
Instructions executing	Yes
Active state	Yes

Important Understanding

Runnable → Running is controlled by CPU scheduler

You cannot directly control this transition.

5. Waiting / Blocked State

Definition: A thread enters Waiting or Blocked state when it is temporarily inactive and waiting for some condition or resource.

Why Thread Goes to Waiting

Reason	Example
sleep() called	Delay execution
join() called	Waiting for another thread
wait() called	Waiting for signal
Resource lock	Waiting for shared resource

Example

```
Thread.sleep(1000);
```

☞ Thread pauses for 1 second

Types of Waiting

1. Timed Waiting

sleep(), wait(time)

2. Indefinite Waiting

wait()

3. Blocked

Waiting for resource (lock)

Key Points

Feature	Explanation
Not executing	Yes
Still alive	Yes
Can return to runnable	Yes

Transition Back

Waiting → Runnable

After:

- time finishes
- resource becomes available
- notify() is called

6. Terminated State

Definition: A thread enters **Terminated state** when it completes execution.

How It Happens

- run() method finishes
- thread stops normally

Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread finished");  
    }  
}
```

Key Points

Feature	Explanation
Execution complete	Yes
Cannot restart	Yes
Thread ends permanently	Yes

Important Rule

Thread cannot be restarted once terminated

7. Complete Execution Flow (Step-by-Step)

1. Thread created → New state
2. start() called → Runnable
3. CPU assigned → Running
4. sleep()/wait() → Waiting/Blocked
5. Back to Runnable
6. Executes again → Running
7. Work complete → Terminated

8. Example Showing Lifecycle Behavior

```
class MyThread extends Thread {  
    public void run() {  
        try {  
            System.out.println("Running");  
            Thread.sleep(1000);  
            System.out.println("Finished");  
        } catch (Exception e) {}  
    }  
}  
  
class Test {  
    public static void main(String[] args) {
```

```
    MyThread t=new MyThread();  
    t.start();  
}  
}
```

Execution Flow

New → Runnable → Running → Waiting → Runnable → Running → Terminated

9. Real-World Example

Online Food App

Thread = Order Processing

New → Order created

Runnable → Waiting for kitchen

Running → Cooking

Waiting → Waiting for delivery

Running → Delivering

Terminated → Order completed

10. Key Understanding

New → Created

Runnable → Ready

Running → Executing

Waiting/Blocked → Paused

Terminated → Finished

11. Important Insights

- Thread does NOT directly go from New → Running
- It must pass through Runnable
- CPU scheduler decides execution
- Waiting state is temporary
- Terminated state is final

Final Mental Model

Thread Life Cycle = Journey of execution

Start → Ready → Running → Pause → Resume → Finish

Unit-6: Creating Threads in Java

Java provides **two standard ways to create threads**:

1. Using **Thread class**
2. Using **Runnable interface**

Understanding both is important because they are used in **different design situations**.

1. Using Thread Class

Concept: In this method, you **create a class that extends the Thread class** and override its run() method.

👉 The run() method contains the code that the thread will execute.

Syntax

```
class MyThread extends Thread {
```

```

    public void run() {
        // task to perform
    }
}

```

Example

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

class Test {
    public static void main(String[] args) {
        MyThread t=new MyThread();
        t.start();
    }
}

```

Execution Flow

Object created → start() called → JVM calls run() → Thread executes

Important Points

Point	Explanation
Must extend Thread class	Required
Override run()	Mandatory
Use start()	To begin execution
Do not call run() directly	It won't create new thread

Why not call run() directly?

t.run(); // ✗ wrong

This will execute like a **normal method**, not a thread.

t.start(); // ✓ correct

Internal Working

start() → creates new thread → calls run() internally

2. Using Runnable Interface

Concept: In this method, you **implement the Runnable interface** and define the run() method.

👉 Then you pass the object to a Thread.

Syntax

```

class MyThread implements Runnable {
    public void run() {
        // task
    }
}

```

Example

```

class MyThread implements Runnable {

```

```

    public void run() {
        System.out.println("Thread running using Runnable");
    }
}
class Test {
    public static void main(String[] args) {
        MyThread m=new MyThread();
        Thread t=new Thread(m);
        t.start();
    }
}

```

Execution Flow

Runnable object → passed to Thread → start() → run() executes

Important Points

Point	Explanation
Implements Runnable	Required
run() method	Must define
Passed to Thread	Required
start() method	Starts thread

Why Runnable is Preferred

Java does not support **multiple inheritance with classes**, but supports it with interfaces.

☞ Runnable allows:

Class can extend another class AND implement Runnable

Example

```

class A {
    void show() {
        System.out.println("Class A");
    }
}
class MyThread extends A implements Runnable {
    public void run() {
        System.out.println("Thread running");
    }
}
class Test {
    public static void main(String[] args) {
        MyThread m=new MyThread();
        Thread t=new Thread(m);
        t.start();
    }
}

```

3. Thread vs Runnable

Detailed Comparison

Feature	Thread Class	Runnable Interface
Type	Class	Interface
Inheritance	Must extend Thread	Can extend other class
Flexibility	Less	More
Code reuse	Limited	Better
Multiple inheritance	Not possible	Possible
Design	Tight coupling	Loose coupling

Deep Understanding

Thread Class → Thread is object itself

Runnable → Task is separate from thread

Concept Difference

Thread Class

Thread = Task + Execution

Runnable Interface

Task (Runnable) + Thread (Executor)

4. Real-World Example

Example: Food Delivery App

Using Thread class

Chef is also delivery person

Not flexible ✗

Using Runnable

Chef cooks

Delivery person delivers

Flexible ✓

5. Multiple Threads Example

```

class MyThread implements Runnable {
    public void run() {
        System.out.println(Thread.currentThread().getName());
    }
}

class Test {
    public static void main(String[] args) {
        MyThread m=new MyThread();
        Thread t1=new Thread(m);
        Thread t2=new Thread(m);
        t1.start();
        t2.start();
    }
}

```

Output (may vary)

Thread-0

Thread-1

6. Important Concepts to Remember

Concept	Explanation
run()	Defines task
start()	Starts thread
Thread scheduler	Decides execution
Multiple threads	Run independently

7. When to Use What

Situation	Use
Simple program	Thread class
Real applications	Runnable
Need flexibility	Runnable
Need inheritance	Runnable

Final Understanding

Thread creation methods:

1. Thread class → simple but less flexible
2. Runnable → preferred, flexible design

Key idea:

Thread = execution

Runnable = task

Unit-6: Thread Methods in Java

Thread methods control **how threads start, execute, pause, coordinate, and identify themselves**. They are essential for writing correct and efficient multithreaded programs.

1. start() Method

Concept: start() is used to **begin execution of a new thread**.

How It Works

start() → JVM creates new thread → calls run() internally

Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread started");  
    }  
}  
  
class Test {  
    public static void main(String[] args) {  
        MyThread t=new MyThread();  
        t.start();  
    }  
}
```

Important Understanding

- start() creates **new thread**
- It automatically calls run()

- Multiple threads can run simultaneously

Common Mistake

`t.run();` // executes like normal method, no new thread

2. run() Method

Concept: `run()` contains the **task that the thread performs**.

Important Point

`run()` defines logic, `start()` executes it in new thread

Example

```
class MyThread extends Thread {  
    public void run() {  
        for(int i=1;i<=3;i++) {  
            System.out.println(i);  
        }  
    }  
}
```

Key Insight

- `run()` can be called directly, but then it behaves like **normal method**
- To achieve multithreading, always use `start()`

3. sleep() Method

Concept

`sleep()` is used to **pause the execution of a thread for a specified time**.

Syntax

`Thread.sleep(milliseconds);`

Example

```
class Test {  
    public static void main(String[] args) {  
        try {  
            for(int i=1;i<=3;i++) {  
                System.out.println(i);  
                Thread.sleep(1000);  
            }  
        } catch(Exception e) {}  
    }  
}
```

Output

```
1  
(wait 1 second)  
2  
(wait 1 second)  
3
```

Key Points

Feature	Explanation
Pauses thread	Yes
Time in milliseconds	Yes
Throws exception	InterruptedException

Important Understanding

sleep() → thread goes to waiting state

4. join() Method

Concept: join() is used to **wait for a thread to complete before continuing execution.**

Why Needed

When one thread depends on another.

Example

```
class MyThread extends Thread {
    public void run() {
        for(int i=1;i<=3;i++) {
            System.out.println("Thread: "+i);
        }
    }
}

class Test {
    public static void main(String[] args) throws Exception {
        MyThread t=new MyThread();
        t.start();
        t.join();
        System.out.println("Main thread resumes");
    }
}
```

Execution Flow

Thread executes → finishes → main thread continues

Key Points

Feature	Explanation
Waits for thread	Yes
Synchronization support	Yes
Useful for sequencing	Yes

5. yield() Method

Concept: yield() is used to **pause current thread temporarily and give chance to other threads.**

Example

```
class MyThread extends Thread {
    public void run() {
        for(int i=1;i<=3;i++) {
            System.out.println(Thread.currentThread().getName());
        }
    }
}
```

```

        Thread.yield();
    }
}
}
class Test {
    public static void main(String[] args) {
        MyThread t1=new MyThread();
        MyThread t2=new MyThread();
        t1.start();
        t2.start();
    }
}

```

Understanding

yield() → suggests scheduler to switch thread

Important Points

Feature	Explanation
Voluntary pause	Yes
Not guaranteed	Scheduler decides
Improves fairness	Yes

6. getName() and setName()

Concept Used to **identify and name threads**.

Example

```

class MyThread extends Thread {
    public void run() {
        System.out.println(Thread.currentThread().getName());
    }
}
class Test {
    public static void main(String[] args) {
        MyThread t=new MyThread();
        t.setName("MyThread1");
        t.start();
    }
}

```

Output

MyThread1

Default Names

Thread-0, Thread-1, Thread-2...

Key Points

Method	Purpose
getName()	Get thread name
setName()	Set thread name

7. isAlive() Method

Concept isAlive() checks whether a thread is **still running or not**.

Syntax

```
t.isAlive();
```

Example

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}

class Test {
    public static void main(String[] args) {
        MyThread t=new MyThread();
        System.out.println(t.isAlive());
        t.start();
        System.out.println(t.isAlive());
    }
}
```

Output

false

true

Meaning

Output	Meaning
false	Thread not started or finished
true	Thread is running

8. Combined Example (All Methods Together)

```
class MyThread extends Thread {
    public void run() {
        try {
            for(int i=1;i<=3;i++) {
                System.out.println(getName()+" "+i);
                Thread.sleep(500);
            }
        } catch(Exception e) {}
    }
}

class Test {
    public static void main(String[] args) throws Exception {
        MyThread t1=new MyThread();
        MyThread t2=new MyThread();
        t1.setName("Thread-1");
        t2.setName("Thread-2");
        t1.start();
    }
}
```

```

        t2.start();
        t1.join();
        t2.join();
        System.out.println("Main thread finished");
    }
}

```

Execution Behavior

Threads run → pause → resume → finish → main thread continues

Final Understanding

start() → starts thread

run() → defines task

sleep() → pauses thread

join() → waits for thread

yield() → gives chance to others

getName()/setName() → identify thread

isAlive() → check thread status

Unit-6: Thread Synchronization & Problems in Multithreading

When multiple threads run together and **access shared data**, problems can occur.

Synchronization is the solution to **control access and maintain correctness**.

6. Thread Synchronization

1. Need for Synchronization

Concept: When multiple threads access the **same resource (shared data)** at the same time, it can lead to **incorrect results**.

☞ Synchronization ensures that **only one thread accesses the resource at a time**.

Real Understanding

Multiple threads → shared resource → conflict → wrong result

Example Without Synchronization

```

class Counter {
    int count=0;
    void increment() {
        count++;
    }
}

class Test {
    public static void main(String[] args) {
        Counter c=new Counter();
        Thread t1=new Thread() -> {
            for(int i=0;i<1000;i++) c.increment();
        };
        Thread t2=new Thread() -> {
            for(int i=0;i<1000;i++) c.increment();
        };
    }
}

```

```

        t1.start();
        t2.start();
        try {
            t1.join();
            t2.join();
        } catch (Exception e) {}
        System.out.println(c.count);
    }
}

```

Expected Output

2000

Actual Output (may vary)

1834, 1970, 1992...

Why This Happens?

count++ is NOT atomic:

Step 1: read value

Step 2: increment

Step 3: write back

Two threads may interfere → **data inconsistency**

Key Idea : Synchronization = control shared resource access

2. synchronized Keyword

Concept: The synchronized keyword is used to **lock a resource so that only one thread can access it at a time.**

Working

Thread enters synchronized section → acquires lock

Other threads → wait

Thread exits → releases lock

Example

```

class Counter {
    int count=0;
    synchronized void increment() {
        count++;
    }
}

```

Important Points

Feature	Explanation
Locking mechanism	Yes
Only one thread allowed	Yes
Prevents data corruption	Yes

3. Synchronized Method

Concept: A method declared as synchronized allows **only one thread at a time** to execute it.

Syntax

```
synchronized void methodName() {  
}
```

Example

```
class Counter {  
    int count=0;  
    synchronized void increment() {  
        count++;  
    }  
}  
  
class Test {  
    public static void main(String[] args) throws Exception {  
        Counter c=new Counter();  
        Thread t1=new Thread() -> {  
            for(int i=0;i<1000;i++) c.increment();  
        };  
        Thread t2=new Thread() -> {  
            for(int i=0;i<1000;i++) c.increment();  
        };  
        t1.start();  
        t2.start();  
        t1.join();  
        t2.join();  
        System.out.println(c.count);  
    }  
}
```

Output

2000

Understanding

Entire method is locked → no conflict → correct result

4. Synchronized Block

Concept: Instead of locking the entire method, we can lock **only a specific part of code**.

Syntax

```
synchronized(object) {  
    // critical section  
}
```

Example

```
class Counter {  
    int count=0;  
    void increment() {  
        synchronized(this) {  
            count++;  
        }  
    }  
}
```

```
}  
}  
}
```

Why Use Block Instead of Method?

Reason	Explanation
Performance	Only critical section locked
Flexibility	Fine control
Efficiency	Less waiting time

Understanding

Only important code is locked → faster execution

7. Problems in Multithreading

1. Race Condition

Concept: A race condition occurs when multiple threads access and modify shared data simultaneously, leading to unpredictable results.

Example

count++;

Two threads:

Thread 1 → read 5

Thread 2 → read 5

Thread 1 → write 6

Thread 2 → write 6

✂ Final result should be 7 but becomes 6

Key Idea

Race = competition between threads

2. Data Inconsistency

Concept : When shared data becomes **incorrect due to improper thread execution**, it is called data inconsistency.

Example

Bank system:

Balance = 5000

Thread 1 withdraws 1000

Thread 2 withdraws 2000

Wrong result:

Balance = 4000 instead of 2000

Cause

- No synchronization
- Multiple updates at same time

Key Idea

Incorrect shared data = data inconsistency

3. Deadlock (Basic Idea)

Concept: A deadlock occurs when two or more threads wait for each other forever, and none can proceed.

Simple Understanding

Thread A → waiting for resource B

Thread B → waiting for resource A

🔒 Both stuck forever

Real Example

Person 1 holds key A, needs key B

Person 2 holds key B, needs key A

🔒 Neither can move

Code Concept

```
synchronized(resource1) {  
    synchronized(resource2) {  
    }  
}
```

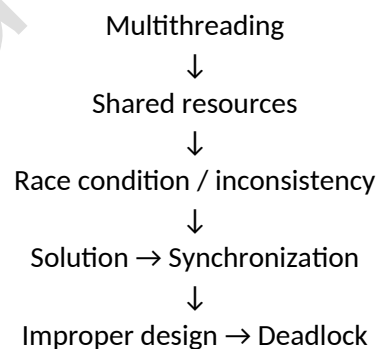
Another thread:

```
synchronized(resource2) {  
    synchronized(resource1) {  
    }  
}
```

Key Points

Feature	Explanation
Infinite waiting	Yes
No execution	System stuck
Hard to detect	Yes

Complete Relationship



Real-World Example (Combined)

Banking System

Multiple users accessing same account

Problems:

- Race condition → wrong balance
- Data inconsistency → incorrect data
- Deadlock → system freeze

Solution:

Synchronization → safe transactions

Final Understanding

Synchronization → controls access

synchronized → lock mechanism

Problems:

Race condition → threads conflict

Data inconsistency → wrong data

Deadlock → threads stuck forever

Goal:

- ✓ Correct execution
- ✓ Safe data
- ✓ Controlled access

Unit-6: Desynchronization & Real-World Applications

This section explains **what happens when synchronization is not used** and how multithreading is applied in **real systems**.

8. Desynchronization

1. Concept of Desynchronization

Definition: Desynchronization means multiple threads access shared resources without any control or coordination.

- ☞ No locking
- ☞ No ordering
- ☞ No safety

Core Idea

Multiple threads → same data → no control → conflict

Deep Understanding

In a multithreaded program:

- Threads run independently
- They may try to access same variable or object
- Without synchronization, operations overlap

☞ This leads to **unpredictable behavior**

Example Situation

Thread 1 → reading data

Thread 2 → writing data

Both run at same time → **data conflict**

2. Without Synchronization

What Happens?

Without synchronization:

- No locking mechanism
- Threads execute freely

- No control over execution order

Internal Problem

Step 1: Read value

Step 2: Modify value

Step 3: Write value

If two threads do this together:

☞ Steps get mixed → incorrect result

Code Example (Without Synchronization)

```
class Counter {
    int count=0;
    void increment() {
        count++;
    }
}

class Test {
    public static void main(String[] args) throws Exception {
        Counter c=new Counter();
        Thread t1=new Thread() -> {
            for(int i=0;i<1000;i++) c.increment();
        };
        Thread t2=new Thread() -> {
            for(int i=0;i<1000;i++) c.increment();
        };
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println(c.count);
    }
}
```

Expected Output

2000

Actual Output

1800 / 1900 / random values

3. Unsafe Execution

Definition: Unsafe execution means program produces incorrect or unpredictable results due to uncontrolled thread access.

Why Unsafe?

Because:

- Threads execute at **different speeds**
- CPU scheduling is **not predictable**
- No guarantee of execution order

Example Scenario

Thread 1 → updates balance

Thread 2 → reads balance

If Thread 2 reads before update completes:

👉 It gets **wrong value**

Key Idea

No synchronization → no safety

4. Example of Issue (Step-by-Step)

Consider:

count++;

Internally

1. Read count (5)

2. Increment (6)

3. Write back (6)

Two Threads Execution

Thread 1 → read 5

Thread 2 → read 5

Thread 1 → write 6

Thread 2 → write 6

Final Result

Expected → 7

Actual → 6

Problem

Lost update → data corruption

5. Data Corruption

Definition: Data corruption occurs when shared data becomes incorrect due to concurrent access without synchronization.

Real Example

Bank Account

Balance = 5000

Two operations:

Thread 1 → withdraw 1000

Thread 2 → withdraw 2000

Wrong Execution

Thread 1 reads 5000

Thread 2 reads 5000

Thread 1 writes 4000

Thread 2 writes 3000

Final Balance

3000 ✗ (Incorrect)

Correct Balance

2000 ✓

Key Idea

Desynchronization → Data corruption

Summary of Desynchronization

No synchronization →

Race condition →

Unsafe execution →

Data corruption

9. Real-World Applications of Multithreading

1. Banking System (Multiple Transactions)

Concept

Banks handle:

- deposits
- withdrawals
- balance checks

simultaneously.

How Threads Are Used

Thread 1 → Deposit

Thread 2 → Withdraw

Thread 3 → Check balance

Problem Without Synchronization

- Wrong balance
- Double withdrawal
- Data inconsistency

With Synchronization

Only one transaction updates account at a time

🔒 Ensures **correct balance**

2. Web Servers (Multiple Users)

Concept: Web servers handle **thousands of users at same time.**

Example

User 1 → Request page

User 2 → Login

User 3 → Download file

Thread Usage

Each request runs as a **separate thread**

Benefits

- Fast response

- Multiple users handled
- Efficient system

Without Multithreading

One request at a time → slow system

3. Operating System (Process Scheduling)

Concept

Operating system manages:

- processes
- threads
- CPU time

How Threads Help

Thread 1 → UI

Thread 2 → background task

Thread 3 → system process

Example

You play music + browse + download file

☞ All tasks run using **threads**

Scheduling

OS decides:

Which thread runs?

How long it runs?

Without Threading

- System becomes slow
- Tasks block each other

4. Combined Real-World System

Example: Online Shopping App

Process → Shopping App

└ Thread 1 → Browse products

└ Thread 2 → Add to cart

└ Thread 3 → Payment

└ Thread 4 → Notifications

Role of Synchronization

Prevent double payment

Ensure correct order details

Final Understanding

Desynchronization:

- ✓ No control
- ✓ Unsafe execution
- ✓ Data corruption

Real-world usage:

- ✓ Banking → safe transactions
- ✓ Web server → multiple users

Key idea: Multithreading powerful but needs synchronization for safety.

dhruvpate/16120